

Архитектура и внутренняя структура клиента World of Warcraft 3.3.5a (Build 12340): Исчерпывающий технический отчет

1. Введение и архитектурный обзор

Настоящий отчет представляет собой углубленное исследование внутренней архитектуры клиента World of Warcraft версии 3.3.5a (сборка 12340), выпущенной Blizzard Entertainment в 2010 году. Целью данного документа является предоставление исчерпывающей технической базы для разработки разделяемой библиотеки на языке C++, способной взаимодействовать с адресным пространством процесса, управлять состоянием игрового персонажа, перехватывать события движка и интегрировать пользовательский интерфейс на базе фреймворка ImGui.

Исследование базируется на анализе статических смещений памяти (offsets), структур данных, таблиц виртуальных функций (VTables) и механизмов взаимодействия подсистем, характерных для архитектуры x86 (32-бит). Клиент версии 3.3.5a, являясь кульминацией дополнения *Wrath of the Lich King*, обладает устоявшейся и хорошо документированной структурой, однако для создания высокоуровневого программного обеспечения требуется детальное понимание не только адресов, но и логики работы ключевых компонентов: *Object Manager* (менеджер объектов), *Lua Engine* (скриптовый движок), *Render Pipeline* (конвейер рендеринга DirectX 9) и *Network Layer* (сетевой слой).

Разработка библиотеки предполагает внедрение (инъекцию) кода в адресное пространство процесса Wow.exe. Это накладывает строгие требования к управлению памятью и потокобезопасности, так как клиент WoW 3.3.5a не является потокобезопасным в контексте вызова большинства своих внутренних функций. Взаимодействие должно происходить либо через перехват основного потока исполнения (Main Thread Hooking), либо через тщательную синхронизацию. В отчете детально рассматриваются методы перехвата функции EndScene интерфейса IDirect3DDevice9 для синхронизации с кадровым циклом игры, что является стандартом де-факто для реализации графических оверлеев ImGui и безопасного исполнения Lua-скриптов.

2. Организация памяти и Object Manager

Центральным элементом архитектуры клиента, отвечающим за хранение и управление

состоянием всех игровых сущностей (игроков, NPC, динамических объектов, предметов), является **Object Manager**. Понимание его структуры — это фундаментальный шаг для любой разработки, связанной с чтением состояния мира.

2.1. Структура менеджера объектов в памяти

В сборке 12340 *Object Manager* не представлен простым массивом. Это сложная структура, базирующаяся на связанных списках (Linked Lists), управление которыми осуществляется через статический указатель на соединение клиента. Доступ к списку объектов требует выполнения цепочки разыменований указателей (pointer dereferencing).

Согласно техническим данным ¹, базовый вход в структуру начинается со статического адреса `StaticClientConnection`. Это глобальная переменная, хранящая указатель на текущую сессию игрока.

Ключевые статические адреса и смещения ¹:

Идентификатор	Адрес / Смещение	Тип данных	Описание
StaticClientConnection	0x00C79CE0	uintptr_t	Глобальный указатель на структуру подключения клиента.
ObjectManagerOffset	0x2ED0	offset	Смещение от ClientConnection до указателя на ObjectManager.
FirstObjectOffset	0xAC	offset	Смещение от ObjectManager до первого объекта в связанном списке (head).
NextObjectOffset	0x3C	offset	Смещение внутри структуры объекта, указывающее на следующий объект

			(next).
LocalPlayerGUIDOf fset	0xC0	offset	Смещение от ObjectManager до GUID локального игрока.

Алгоритм итерации по объектам

Для формирования списка видимых сущностей библиотека C++ должна реализовать итератор, проходящий по односвязному списку. Логика обхода выглядит следующим образом:

1. **Получение базового адреса:** Считывается значение по адресу 0x00C79CE0. Если значение равно 0, клиент не находится в игровом мире.
2. **Переход к менеджеру:** К полученному адресу добавляется смещение 0x2ED0. По этому адресу располагается указатель на экземпляр класса ObjectManager (часто именуемый CurMgr в сообществе реверс-инжиниринга).
3. **Захват головы списка:** Из структуры менеджера по смещению 0xAC считывается адрес первого объекта (FirstObject).
4. **Циклический обход:**
 - Проверяется валидность указателя на текущий объект (он не должен быть NULL и должен быть выровнен по границе 4 байт, то есть младший бит равен 0).
 - Производятся необходимые операции с объектом (чтение типа, координат, дескрипторов).
 - Считывается адрес следующего объекта, расположенный по смещению 0x3C относительно текущего объекта.
 - Цикл повторяется до тех пор, пока указатель на следующий объект не станет равным 0.

Важно отметить, что структура списка объектов в 3.3.5a высоко динамична. Объекты могут создаваться и уничтожаться в каждом кадре. Попытка обхода этого списка из внешнего потока без приостановки основного потока игры может привести к состоянию гонки (Race Condition) и аварийному завершению работы (Access Violation), если объект будет удален во время чтения. Рекомендуется выполнять обход внутри перехваченной функции EndScene.

2.2. Типология объектов и наследование

Каждый объект в памяти наследуется от базового класса CObject. По смещению 0x14 внутри структуры объекта находится идентификатор типа (TypeID), определяющий, какие данные доступны для этого объекта.¹

Перечисление типов объектов (TypeID):

ID	Тип (Enum)	C++ Класс (Условный)	Описание
1	Item	CGItem	Предметы в инвентаре или экипировке.
2	Container	CGContainer	Сумки, рюкзаки (наследуется от Item).
3	Unit	CGUnit	NPC, монстры, питомцы.
4	Player	CGPlayer	Игровые персонажи (наследуется от Unit).
5	GameObject	CGGameObject	Интерактивные объекты мира (сундуки, двери, руда, трава).
6	DynamicObject	CGDynamicObject	Визуальные эффекты заклинаний, тотемы.
7	Corpse	CGCorpse	Тела умерших игроков или NPC.

Для разработчика библиотеки критически важно различать типы Unit и Player. Хотя они разделяют множество общих полей (здоровье, мана, позиция), Player содержит уникальные дескрипторы, связанные с системой опыта, репутации и навыков. В C++ коде это обычно реализуется через приведение типов (casting) на основе значения по смещению 0x14.

2.3. Система дескрипторов (UnitFields)

Вся игровая статистика сущности хранится не в полях класса C++, а в специальном динамическом массиве, называемом "Дескрипторами" или UnitFields. Указатель на этот массив всегда находится по смещению 0x8 от начала объекта.¹

Доступ к конкретному параметру (например, текущему здоровью) осуществляется по индексу в этом массиве. Поскольку массив хранит 32-битные значения (uint32 или float), физическое смещение вычисляется как $\text{Index} * 4$.

Карта дескрипторов для Unit и Player ¹:

Параметр	Индекс (Hex)	Смещение (Байты)	Тип данных	Примечание
OBJECT_FIELD_GUID	0x0	0x00	uint64	Полный 64-битный GUID объекта (занимает 2 слота: 0x0 и 0x1).
OBJECT_FIELD_TYPE	0x2	0x08	uint32	Битовая маска типа объекта.
OBJECT_FIELD_ENTRY	0x3	0x0C	uint32	ID записи в базе данных (NPC ID).
UNIT_FIELD_HEALTH	0x18	0x60	uint32	Текущее здоровье.
UNIT_FIELD_MAXHEALTH	0x20	0x80	uint32	Максимальное здоровье.
UNIT_FIELD_POWER1	0x19	0x64	uint32	Текущая мана/ярость/энергия.
UNIT_FIELD_MAXPOWER1	0x21	0x84	uint32	Максимум

AXPOWER1				маны/ярости.
UNIT_FIELD_LEVEL	0x36	0xD8	uint32	Уровень персонажа/NP C.
UNIT_FIELD_F ACTIONTEMPLATE	0x37	0xDC	uint32	ID фракции (для определения враждебности).
UNIT_FIELD_FLAGS	0x39	0xE4	uint32	Флаги состояния (бой, оглушение, полет).
UNIT_FIELD_SUMMONEDBY	0xE	0x38	uint64	GUID владельца (для питомцев/тотемов).

Для объектов типа Player массив дескрипторов продолжается после полей Unit. Точные индексы для специфичных полей игрока (например, XP) начинаются после последнего индекса Unit.

Позиционирование в пространстве

В отличие от дескрипторов, координаты объекта хранятся непосредственно в структуре класса CGameObject (или его наследников), а не в массиве UnitFields. Это сделано для оптимизации рендеринга и физических расчетов.

Смещения координат ¹:

- **Pos X:** 0x79C (float)
- **Pos Y:** 0x798 (float)
- **Pos Z:** 0x7A0 (float)
- **Rotation (Facing):** 0x7A8 (float, в радианах)

Эти значения доступны для записи, но прямая их модификация ("телепортация") часто

отслеживается античит-системами на серверной стороне. Для легитимного перемещения используется подсистема Click-To-Move.

3. Подсистема навигации и управления (Click-To-Move)

Для программного управления персонажем в версии 3.3.5a наиболее эффективным методом является использование механизма **Click-To-Move (CTM)**. Эта функция, изначально разработанная для удобства управления мышью, позволяет задавать точные 3D-координаты назначения и тип действия, что идеально подходит для создания ботов.

3.1. Структура CTM и адресация

CTM-структура располагается в памяти по фиксированным смещениям относительно базового класса игрока или как глобальная структура. В контексте взаимодействия с функцией `CGPlayer_C::ClickToMove` (адрес `0x00611130` согласно ² и общей базе знаний 3.3.5a), структура передается как аргумент или заполняется перед вызовом.

Адрес функции `ClickToMove`: `0x00611130`.² В сообществе разработчиков для 3.3.5a часто используется запись значений напрямую в память структуры CTM, адрес которой обычно связывается с базовым адресом игрока плюс смещение, либо находится по глобальному адресу `0x00CA11D0` (для некоторых версий клиента 3.3.5, требует уточнения через сканирование паттернов вокруг функции `0x00611130`).

Поля структуры `ClickToMove` ³:

Смещение	Тип	Имя поля	Описание
0x04	float	TurnScale	Скорость/масштаб поворота.
0x0C	float	Distance	Дистанция взаимодействия (насколько близко подойти).
0x1C	int	Action	Тип выполняемого действия (Enum).

0x20	uint64	InteractGUID	GUID цели для взаимодействия (атака, лут, разговор).
0x74	float	DestX	Целевая координата X.
0x78	float	DestY	Целевая координата Y.
0x7C	float	DestZ	Целевая координата Z.

3.2. Типы действий (Action IDs)

Правильный выбор Action ID определяет поведение клиента при достижении точки назначения или при немедленном выполнении команды.

- **FaceTarget (0x1):** Поворачивает персонажа лицом к указанным координатам или GUID. Не вызывает перемещения. Используется для ориентации перед кастом заклинаний, требующих, чтобы цель была перед лицом.
- **Stop (0x3):** Немедленно останавливает движение. Полезно для прерывания авто-бега или отмены предыдущей команды CTM.
- **Move (0x4):** Стандартное перемещение к координатам (X, Y, Z). Клиент построит путь (насколько позволяет локальный навмеш, если он есть, или по прямой) к точке.
- **NpcInteract (0x5):** Движение к NPC (указанному в InteractGUID) и взаимодействие с ним (открытие окна диалога, банка, аукциона).
- **Loot (0x6):** Специализированное действие для лута трупов мобов.
- **ObjInteract (0x7):** Взаимодействие с игровыми объектами (GameObjects), такими как рудные жилы, трава, почтовые ящики.
- **AttackGuid (0xA):** Атака ближнего боя. Персонаж будет следовать за целью (InteractGUID) и пытаться нанести удар автоатакой.
- **AttackPos (0xB / 0xD):** Атака в точку (реже используется).

3.3. Математика навигации и поворота

Даже при использовании CTM, библиотеке часто требуется самостоятельно рассчитывать углы поворота, например, для проверки, смотрит ли персонаж на цель.

Угол поворота (Rotation) в WoW хранится в радианах (от 0 до 2π). Север обычно

соответствует 0 или π , в зависимости от системы координат.

Для вычисления угла поворота к точке (Tx, Ty) от позиции игрока (Px, Py) используется функция арктангенса:

$$\theta = \text{atan2}(Ty - Py, Tx - Px)$$

Полученное значение θ необходимо нормализовать в диапазон $[0, 2\pi]$, если результат отрицательный (добавить 2π).

4. Интеграция с графическим конвейером (DirectX 9 и ImGui)

Реализация внутриигрового интерфейса (UI) с использованием библиотеки **ImGui** требует внедрения в процесс рендеринга (Hooking). WoW 3.3.5a использует API DirectX 9.

4.1. Механизм перехвата (Hooking) EndScene

Для отрисовки графики поверх игры необходимо выполнить код в момент, когда игра закончила формирование 3D-сцены, но еще не отправила кадр на монитор. Идеальным местом для этого является функция EndScene интерфейса IDirect3DDevice9.

Адрес этой функции не является статическим, так как он зависит от загруженной DLL драйвера DirectX. Поэтому используется метод перехвата через таблицу виртуальных методов (VTable Hooking).

1. **Поиск VTable:** Библиотека создает фиктивный ("dummy") объект IDirect3D9 и IDirect3DDevice9 при инициализации. Это позволяет получить валидный указатель на VTable устройства, не вмешиваясь в основной рендер на этапе старта.
2. **Индексы VTable:**
 - **EndScene:** Индекс **42**.
 - **Reset:** Индекс **16**. (Необходим для обработки изменения размера окна или Alt-Tab, иначе игра "крашнется" при потере устройства).
3. **Установка хука:** Оригинальный адрес функции по индексу 42 подменяется на адрес функции-обертки Hooked_EndScene внутри нашей DLL. Оригинальный адрес сохраняется для вызова в конце обертки (Trampoline).

4.2. Реализация ImGui и обработка ввода (WndProc)

Внутри Hooked_EndScene происходит инициализация и рендеринг ImGui.

C++

```
// Псевдокод логики EndScene
HRESULT __stdcall Hooked_EndScene(LPDIRECT3DDEVICE9 pDevice) {
    if (!g_Initialized) {
        ImGui::CreateContext();
        ImGui_ImplDX9_Init(pDevice);
        g_Initialized = true;
    }

    ImGui_ImplDX9_NewFrame();
    // Отрисовка окон ImGui
    ImGui::Render();
    ImGui_ImplDX9_RenderDrawData(ImGui::GetDrawData());

    return oEndScene(pDevice); // Вызов оригинальной функции
}
```

Однако ImGui требует ввода от мыши и клавиатуры. Для этого необходимо перехватить оконную процедуру WndProc (Window Procedure) игрового окна. Используя WinAPI функцию SetWindowLongPtr с параметром GWLP_WNDPROC, библиотека подменяет обработчик сообщений Windows. Новый обработчик передает события ввода (нажатия клавиш, движение мыши) в ImGui_ImplWin32_WndProcHandler. Если ImGui "захватил" ввод (например, открыто меню бота), событие блокируется и не передается игре, что позволяет кликать по меню, не поворачивая камеру в игре.

4.3. Проекция World-To-Screen (W2S)

Для реализации функционала ESP (Box ESP, линии до целей) необходимо проецировать 3D-координаты объектов мира на 2D-плоскость экрана монитора. Для этого используются матрицы камеры.

Согласно ³, данные камеры находятся в динамической памяти, но доступ к ним осуществляется через указатель, возвращаемый функцией CGWorldFrame__GetActiveCamera (окрестности адреса 0x007A07E4 ⁷).

Математика проекции:

Координата в пространстве экрана V_{screen} получается путем умножения координаты

мира V_{world} на видовую проекционную матрицу (ViewProjection Matrix):

$$V_{clip} = V_{world} \times M_{view_proj}$$

После умножения производится перспективное деление (Perspective Divide):

$$V_{ndc}.x = V_{clip}.x / V_{clip}.w$$

$$V_{ndc}.y = V_{clip}.y / V_{clip}.w$$

Если $V_{clip}.w < 0$ (или малое значение ϵ), точка находится позади камеры и не должна отрисовываться.

Смещения данных камеры ⁵:

- **Camera X/Y/Z:** Смещения 0x8, 0xC, 0x10.
- **Матрица вида (View Matrix):** Обычно это массив из 16 float значений (Matrix4x4), расположенный в памяти камеры. В некоторых случаях требуется перемножить матрицу вида и матрицу проекции вручную, если игра не хранит готовую ViewProj матрицу. Смещения матрицы часто начинаются с 0x14 или вычисляются динамически.

5. Интеграция с Lua Engine

World of Warcraft 3.3.5a использует модифицированную версию Lua 5.1. Интеграция с Lua позволяет вызывать внутриигровые API (например, CastSpellByName) и получать данные, недоступные через Object Manager (например, содержимое чата).

5.1. Исполнение скриптов (FrameScript_Execute)

Для выполнения произвольного Lua-кода из C++ используется функция, известная как FrameScript::Execute или Lua_DoString.

Адрес функции: 0x00819210.⁸

Эта функция принимает строку с кодом Lua, имя файла (контекст, обычно "GlueXML" или "RunScript") и указатель на дополнительное состояние.

Вызов этой функции позволяет библиотеке делать всё, что доступно макросам /script.

Проблема Protected Lua:

Многие мощные функции (каст заклинаний, движение) защищены ("Protected") и могут вызываться только в ответ на аппаратное событие (клик мыши). Обход этой защиты в 3.3.5a

возможен путем патчинга функции проверки безопасности (IsSecure), заставляя её всегда возвращать true. Это открывает доступ к полному API, известному как "Lua Unlock".

5.2. Получение данных (FrameScript_GetText)

Для чтения значений глобальных переменных Lua (например, текста в окне) используется функция FrameScript_GetText (или Lua_GetLocalizedText).

Адрес функции: 0x007225E0.⁸

Также критически важен доступ к самому состоянию Lua (lua_State). Глобальный указатель на lua_State находится по адресу 0x00D3F78C.⁸ Имея этот указатель, можно использовать стандартные функции C API Lua (lua_gettop, lua_tostring, lua_pushstring) для прямого манипулирования стеком Lua, что быстрее и надежнее парсинга строк.

5.3. Перехват событий (Events)

События игры (например, COMBAT_LOG_EVENT_UNFILTERED) обрабатываются через систему сигналов. Перехват функции FrameScript_SignalEvent позволяет C++ библиотеке реагировать на события быстрее, чем это делает UI игры. Это позволяет, например, мгновенно использовать прерывание заклинания (Kick), как только сервер присылает пакет о начале каста противником, не дожидаясь отрисовки каст-бара.

6. Сетевой уровень и Opcodes

Наиболее продвинутый уровень взаимодействия — работа с сетевыми пакетами. Это позволяет отправлять команды серверу напрямую, минуя ограничения интерфейса Lua и проверки клиента.

6.1. Структура пакетов и Opcodes

Коммуникация строится на обмене пакетами, идентифицируемыми кодами операций (Opcodes). Список опкодов для 3.3.5a (12340) огромен.⁹

Критически важные Opcodes:

- CMSG_CAST_SPELL.⁹
- CMSG_USE_ITEM (0x00AB): Использование предмета.
- CMSG_GOSSIP_HELLO (0x017B): Разговор с NPC.
- SMSG_UPDATE_OBJECT (0x00A9): Основной пакет от сервера, содержащий информацию о появлении новых объектов или изменении их полей. Разбор этого пакета позволяет обновлять данные в Object Manager.

6.2. Отправка пакетов (NetClient::SendPacket)

Для отправки пакета необходимо сформировать объект класса CDataStore (буфер данных) и вызвать функцию отправки.

Адрес функции SendPacket: 0x005379A0.² Примечание: Источник ² имеет в названии 1.12.1, но контекст обсуждения и смещения указывают на применимость к 3.x ветке в ряде случаев, либо на схожесть паттернов. Для 3.3.5a стандартным адресом отправки является функция, вызываемая внутри ClientServices::Send. Наиболее надежный метод поиска — найти ссылку на строку "CDataStore::Finalize" или использовать VTable объекта NetClient.

При отправке пакетов типа CMSG (Client Message) клиент 3.3.5a автоматически шифрует заголовок пакета (но не тело), используя потоковый шифр RC4 и сессионный ключ, согласованный при логине. Вызов внутренней функции SendPacket предпочтительнее самостоятельной реализации шифрования, так как она уже использует правильное состояние RC4.

7. Сводная таблица данных для разработчика

Ниже приведена консолидированная таблица адресов и смещений, необходимых для реализации библиотеки, на основе всех проанализированных данных.¹

Категория	Имя	Адрес / Смещение (Hex)	Источник	Описание
ObjectMgr	StaticClientConnection	0x00C79CE0	¹	Глобальный указатель подключения.
ObjectMgr	ObjectManagerOffset	0x2ED0	¹	Смещение к менеджеру объектов.
ObjectMgr	FirstObject	0xAC	¹	Начало списка объектов.
ObjectMgr	NextObject	0x3C	¹	Указатель на следующий

				объект.
ObjectMgr	LocalPlayerGUID	0x00BD07A8	1	Статический адрес GUID игрока.
ObjectMgr	LocalTargetGUID	0x00BD07B0	1	Статический адрес GUID цели.
Unit	DescriptorsOffset	0x8	1	Смещение массива дескрипторов.
Unit	Pos_X	0x79C	1	Координата X.
Unit	Pos_Y	0x798	1	Координата Y.
Unit	Pos_Z	0x7A0	1	Координата Z.
Unit	Rotation	0x7A8	1	Угол поворота.
CTM	CTM_Base	0x00CA11D0	4	Базовый адрес структуры CTM (ориентировочно).
CTM	CTM_Func	0x00611130	2	Функция ClickToMove.
CTM	ActionType	0x1C	3	Тип действия (смещение в структуре).
CTM	Dest_X	0x74	3	Цель

				движения X.
Lua	FrameScript_Execute	0x00819210	8	Выполнение Lua кода.
Lua	FrameScript_GetText	0x007225E0	8	Получение текста/переменных.
Lua	Lua_State	0x00D3F78C	8	Указатель на состояние Lua.
Network	SendPacket	0x005379A0	2	Функция отправки пакета.
Camera	GetActiveCamera	0x007A07E4	7	Функция получения камеры (контекст ошибки).

8. Заключение

Разработка библиотеки для World of Warcraft 3.3.5a (12340) требует системного подхода к управлению памятью и понимания взаимосвязей между подсистемами клиента.

Представленные данные обеспечивают полный набор инструментов для создания функционала: от пассивного чтения состояния мира через Object Manager и дескрипторы до активного управления через Click-To-Move и инъекцию Lua-кода.

Особое внимание при реализации следует уделить безопасности: минимизации записи в память (использовать только CTM и Lua, избегая патчинга кода .text секции во избежание детекта Warden), а также корректной синхронизации с основным потоком игры через хук EndScene для обеспечения стабильности работы графического интерфейса ImGui.

Комбинирование этих методов позволяет создать мощный инструмент автоматизации и расширения функционала клиента, сохраняя при этом высокую производительность и стабильность приложения.

Works cited

1. WoW-Object-Manager/WoWObjMgr/PlayerScan.cs at master - GitHub, accessed

February 13, 2026,

<https://github.com/johnmoore/WoW-Object-Manager/blob/master/WoWObjMgr/PlayerScan.cs>

2. FishBot 1.12.1 - GitHub, accessed February 13, 2026,
<https://github.com/WowDevs/Fishbot-1.12.1/blob/master/FishBot%201.12.1/Information%20Library%20for%201.12.1/1.12.1%20Stuff>
3. Click To Move - WowDev wiki, accessed February 13, 2026,
https://wowdev.wiki/Click_To_Move
4. AzDeltaQQ/WotLKRotations: A Python-based experimental framework for interacting with World of Warcraft (3.3.5a - 12340 client) memory to monitor game state and potentially execute combat rotations. (Extremely Work-in-progress) - GitHub, accessed February 13, 2026,
<https://github.com/AzDeltaQQ/WotLKRotations>
5. Camera - WowDev wiki, accessed February 13, 2026, <https://wowdev.wiki/Camera>
6. Why is my FOV Perspective Matrix4x4 looking Orthographic (DotNet and SkiaSharp)?, accessed February 13, 2026,
<https://stackoverflow.com/questions/78727172/why-is-my-fov-perspective-matrix-4x4-looking-orthographic-dotnet-and-skiasharp>
7. Client Error Analysis - WowDev wiki, accessed February 13, 2026,
https://wowdev.wiki/Client_Error_Analysis
8. [WoW][3.3.5.12340] Info Dump Thread - OwnedCore, accessed February 13, 2026,
<https://www.ownedcore.com/forums/world-of-warcraft/world-of-warcraft-bots-programs/wow-memory-editing/300463-wow-3-3-5-12340-info-dump-thread.html>
9. WowPacketParser/WowPacketParser/Enums/Version/V4_0_3_13329/Opcodes.cs at master · TrinityCore/WowPacketParser - GitHub, accessed February 13, 2026,
https://github.com/TrinityCore/WowPacketParser/blob/master/WowPacketParser/Enums/Version/V4_0_3_13329/Opcodes.cs